

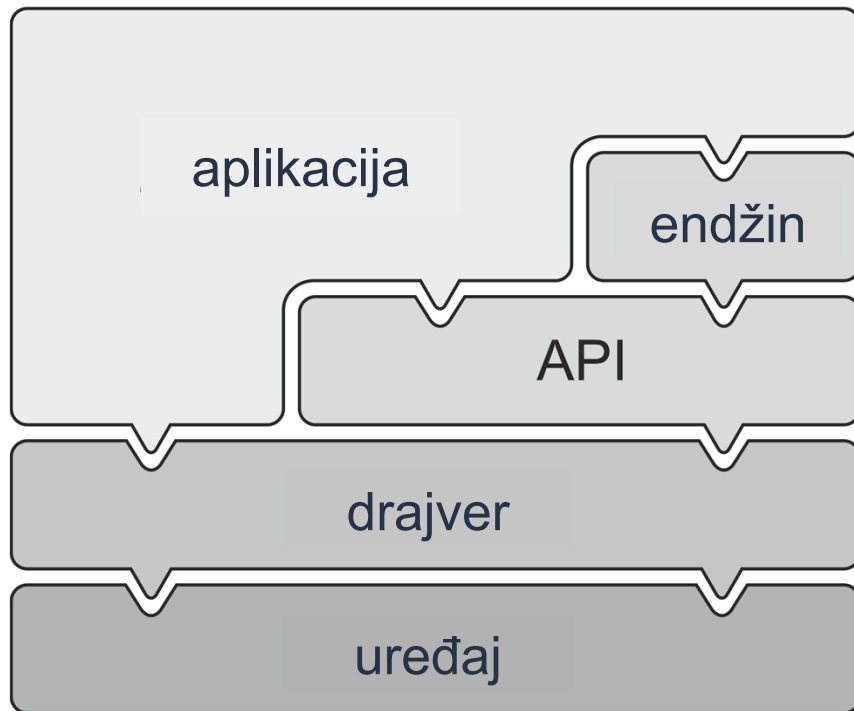
Računarska grafika
(3OER7O02)

**Dvodimenzionalna računarska
grafika primenom GDI**

Vežbe



Nivoi apstrakcije u pristupu uređaju



GDI (*Graphics Device Interface*)

- 2D grafički API, koji je razvila kompanija Microsoft za svoje Windows operative sisteme
- Izvorno napisan za 16-bitne verzije Windowsa, ali je ažuriran za 32-bitne i 64-bitne verzije
- Dizajniran prvenstveno za C/C++ programere

Crtanje nezavisno od uređaja

Sofrverska podrška za crtanje nezavisno od uređaja ostvarena je kroz dva DLLa:

- **Gdi.dll** (*graphics device interface*) i

- **drajvera uređaja** (zavisi od konkretnog izlaznog uređaja, npr. Epson9.dll)

Aplikacija mora da obavesti GDI koji drajver treba da učitava i da pripremi uređaj za sam proces crtanja (postavlja debljinu i boju linija, četku, paletu, font, *clip-region*, itd.). Ovi zadaci se ostvaruju kreiranjem i upravljanjem kontekstom uređaja (eng. **device context** - DC).

Konteks uređaja

Kontekst uređaja

Kontekst uređaja (eng. *Device Context* – DC) je osnovna komponenta preko koje aplikacija pristupa odgovarajućem uređaju.

„Ono što je slikarima platno, to je za Windows programere Device Context, ali je DC i mnogo više od toga.“

Kontekst uređaja je struktura podataka koja definiše skup grafičkih objekata i njima dodeljenih atributa, kao i grafičke režime koji utiču na prikaz.

Tipovi konteksta uređaja

Postoji četiri tipa konteksta uređaja:

- **prikazni** (eng. *Display*) – podržava prikaz na video-uređaju, kao što je ekran,
- **printerski** (eng. *Printer*) – podržava iscrtavanje na štampačima i ploterima,
- **memorijski** (eng. *Memory*) – podržava iscrtavanje u memoriji, najčešće kao pripremu za prikaz na nekom drugom uređaju i
- **informativni** (eng. *Information*) – podržava pribavljanje podrazumevanih postavki konteksta za neki tip uređaja.

Kako pribaviti DC prozora

- Preuzimanjem *handle*-a na klijentsku površinu prozora (Win32):

HDC GetDC(HWND *hWnd* // handle to a window);*

- Preuzimanjem *handle*-a na klijentsku površinu prozora (MFC):

CDC* CWnd::GetDC(); *!

* Nakon završetka rada osloboditi sa **ReleaseDC()**, osim u specijalnim slučajevima!

! U MFC-u, funkcije za rad sa DC-jem inkapsulirane su u klasi **CDC**.

U MFC Document-View aplikacijama

```
void CP1View::OnDraw(CDC* pDC)
{
    CP1Doc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);
    // TODO: add draw code for native data here
}
```

Komponente konteksta uređaja

■ Atributi

- ▷ boja teksta i pozadine

■ Grafički modovi

- ▷ background, drawing, mapping, polygon-fill, stretching

■ Grafički objekti

- ▷ olovke, četke, bitmape, fontovi, putanje, regioni,...

Atributi DC-ja

Boje teksta i pozadine

- **Text color** – boja kojom se ispisuje tekst, ali ne samo to (*foreground color*)
 - `COLORREF CDC::GetTextColor( );`
 - `COLORREF CDC::SetTextColor( COLORREF crColor );`
- **Background color** – boja pozadine (iza teksta, ali i bilo kog drugog objekta)
 - `COLORREF CDC::GetBkColor( );`
 - `COLORREF CDC::SetBkColor( COLORREF crColor );`

Definisanje boje

- **COLORREF** – 32bit vrednost koja definiše RGB boju u formatu **0x00**bb****gg****rr****
- **RGB** – makro koji olakšava kreiranje boja

COLORREF RGB(

BYTE *bRed*,

BYTE *bGreen*,

BYTE *bBlue*);

RGB(255,0,0)

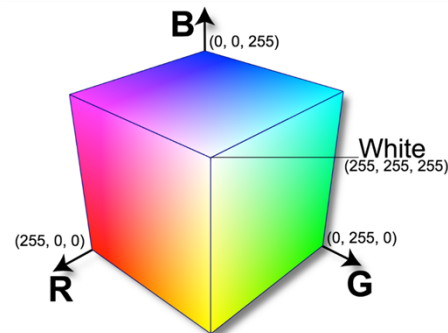
RGB(0,0,255)

RGB(255,0,255)

RGB(0,255,0)

RGB(255,255,0)

RGB(0,255,255)



```
#define RGB(r, g, b)
```

```
((DWORD) (((BYTE) (r) | ((WORD) (g) << 8)) | (((DWORD) (BYTE) (b)) << 16)))
```

Ekstrakcija komponenti boja

Iz 32-bitne RGB (COLORREF) vrednosti pojednih komponentata (kanala) mogu se izdvojiti ručno (maskiranjem i šiftovanjem), ali je lakše korišćenjem makroa (GetRValue, GetGValue i GetBValue)

```
#define GetRValue(rgb) ((BYTE) (rgb))
```

```
#define GetGValue(rgb) ((BYTE) (((WORD) (rgb)) >> 8))
```

```
#define GetBValue(rgb) ((BYTE) ((rgb) >> 16))
```

Grafički modovi

Win32 API podržava 5 grafičkih modova:

- ***background* mod** – definiše način mešanja pozadinske boje sa drugim bojama bitmapa ili teksta,
- ***drawing* mod** - definiše način mešanja *foreground* boje sa drugim bojama olovaka, četki, bitmapa ili teksta,
- ***mapping* mod** – definiše kako se koordinate mapiraju iz logičkih u fizičke,
- ***polygon-fill* mod** – definiše kako se šablon četke koristi za ispunu kompleksnih regiona i
- ***stretching* mod** – definiše kako se boje bitmape mešaju sa drugim bojama u prozoru kada je bitmapa kompresovana ili smanjena

Snimanje/vraćanje stanja DC-ja

■ virtual int CDC:: SaveDC();

- svi selektovani objekti i postavljeni grafički modovi snimaju se u *context* steku

■ BOOL CDC::RestoreDC(int nSavedDC);

- *nSavedDC* određuje instancu DC koja se vraća iz setka:
 - pozitivan – odgovarajuća instanca (prosleđuje se vrednost koju je vratio prethodni poziv funkcije SaveDC),
 - negativan – relativno u odnosu na vrh steka (-1 poslednje snimljeno stanje)
- Sve instance DC-ja između izabranog indeksa i vrha steka se brišu!

GDI+

Šta je potrebno za GDI+

- `gdiplus.dll` – već je uključen u distribuciju WinXP, ali ga stariji OS ne sadrže
- `GdiPlus.lib` – statička biblioteka
- Include datoteke – `GdiPlus.h`, `GdiPlusBase.h`, `GdiPlusBitmap.h` ...

StdAfx.h ili View.cpp

```
#include <gdiplus.h>  
#pragma comment(lib, "\\GDIPlus\\GdiPlus.lib")  
using namespace Gdiplus;
```

Ako nije definisan ULONG_PTR dodati i sledeće

- #ifndef ULONG_PTR
- #define ULONG_PTR unsigned long*
- #endif

Application.h / .cpp

```
ULONG_PTR m_gdiplusToken;
```

```
CApp::InitInstance()
```

```
■ Gdiplus::GdiplusStartupInput gdiplusStartupInput;
```

```
■ Gdiplus::GdiplusStartup(&m_gdiplusToken,  
    &gdiplusStartupInput, NULL);
```

```
CApp::ExitInstance()
```

```
■ Gdiplus::GdiplusShutdown(m_gdiplusToken);
```

Formiranje Graphics objekta

- Ono što je DC za GDI, to je **Graphics** za GDI+
- Može se kreirati na više načina, a korist ćemo sledeća 2:
 - ▷ **Graphics(HDC *hdc*);**
 - ▷ **Graphics(Image* *image*);**

Primer:

```
void CGDIPlusView::OnDraw(CDC* pDC)
{
    Graphics graphics(pDC->m_hDC);
}
```

Pitanja

